I cheated here in one respect—the have_selector matcher is not part of core RSpec; it's actually part of Capybara, but you've already loaded Capybara, so that's fine. But have_selector is way better than the core way of querying DOM trees, so it's worth including here.

What does this view test do?

First you create your given: the data. Use let to create the on-schedule project and task. The objects need to be in the database so that the Rails associations all work. But there are other choices here that might make the test faster, including creating projects and stubbing the on_schedule? method or using FactoryBot.build_stubbed to create real objects without having to save them in the database.

In the spec itself, you set the @projects variable. You're testing the view in isolation, so you don't have a controller to set this up for you; you need to create the instance variables that will be given to the view.

Our "when" section is one line, render, which tells RSpec to render the view. Which view? The one specified by the outermost describe block. In this case, that's projects/index—Rails will connect that to the index.html.erb file in the file base. Alternatively, you can explicitly pass the view as an argument, render template: "projects/index". If the view is a partial, you need to specify that: render partial: "projects/data_row". An optional locals argument with key/value pairs specifies local variables to the partial. As with the regular Rails render method, you can use a shortcut of the form render "projects/data_row, project: @project".

All Rails helpers are loaded. If you want to stub one of their values, the helper methods are accessible via a view object, as in view.stub(:current_user).and_return(User.new). You can also use stub_template to stub a partial that you don't want to render; stub_template takes a key/value pair where the key is the exact file name of the partial and the value is the string you want returned in place of rendering the partial.

In the "then" portion of the spec, the rendered text is available via the method rendered. You can then use any RSpec matcher to set expectations on that value. In this test you're using have_selector, but you can also use match to do a simple regular-expression match.

In this case you're testing for the existence of a selector pattern with a DOM ID #project_#{on_schedule.id} that has a subordinate object containing the DOM class.on_schedule, and a similar selector pattern with the DOM ID #project_#{behind_schedule.id} containing an item with the DOM class .behind_schedule. You're passing the result of the rendered view to the matcher.